# Ejercicio 8 – Parte 2 (puntos 2, 3 y 4)

2) Pruebas paso a paso:

| | Zona 1 | Zona 2 | Zona 3 | Zona 4 |
|---|---|---|---|---|
| Zona 1 | x | 2 | 5 | 1 |
| Zona 2 | 2 | x | 1 | 3 |
| Zona 3 | 5 | 1 | x | 2 |
| Zona 4 | 1 | 3 | 2 | x |

*Figura 1. Escenario reducido de 4 zonas.*

### Inicialización

| Variable | Valor después de inicializar |
|---|---|
| `inicio` | Z1 (el origen existe, no se devuelve nulo) |
| `U` | {Z1} |
| `VmenosU` | {Z2, Z3, Z4} |
| `arbol` | vértices {Z1, Z2, Z3, Z4}, sin aristas |

### Iteración 1 del MIENTRAS (VmenosU no está vacío)

| u | Arista revisada | ¿w ∈ V? | Peso | Decisión → mejor / mejorPeso |
|---|---|---|---|---|
| Z1 | Z1–Z2 | sí | 2 | 2 < ∞ → Z1→Z2 / 2 |
| Z1 | Z1–Z3 | sí | 5 | 5 < 2 no → sigue Z1→Z2 / 2 |
| Z1 | Z1–Z4 | sí | 1 | 1 < 2 → Z1→Z4 / 1 |

Devuelve **Z1→Z4 (1)**. Se agrega al árbol, Z4 pasa a U. → U = {Z1, Z4}, VmenosU = {Z2, Z3}.

### Iteración 2

Se llama a `searchMinEdge(g, {Z1, Z4}, {Z2, Z3})`.

| u | Arista revisada | ¿w ∈ V? | Peso | Decisión → mejor / mejorPeso |
|---|---|---|---|---|
| Z1 | Z1–Z2 | sí | 2 | 2 < ∞ → Z1→Z2 / 2 |
| Z1 | Z1–Z3 | sí | 5 | 5 < 2 no |
| Z1 | Z1–Z4 | no (Z4 ∈ U) | — | se ignora |
| Z4 | Z4–Z1 | no (Z1 ∈ U) | — | se ignora |
| Z4 | Z4–Z2 | sí | 3 | 3 < 2 no |
| Z4 | Z4–Z3 | sí | 2 | 2 < 2 no (empate: se queda la primera encontrada) |

Devuelve **Z1→Z2 (2)**. → U = {Z1, Z4, Z2}, VmenosU = {Z3}.

### Iteración 3

Se llama a `searchMinEdge(g, {Z1, Z4, Z2}, {Z3})`.

| u | Arista revisada | ¿w ∈ V? | Peso | Decisión → mejor / mejorPeso |
|---|---|---|---|---|
| Z1 | Z1–Z2 | no | — | se ignora |
| Z1 | Z1–Z3 | sí | 5 | 5 < ∞ → Z1→Z3 / 5 |

| u | Arista revisada | ¿w ∈ V? | Peso | Decisión → mejor / mejorPeso |
|---|---|---|---|---|
| Z1 | Z1–Z4 | no | — | se ignora |
| Z4 | Z4–Z1 | no | — | se ignora |
| Z4 | Z4–Z2 | no | — | se ignora |
| Z4 | Z4–Z3 | sí | 2 | 2 < 5 → Z4→Z3 / 2 |
| Z2 | Z2–Z1 | no | — | se ignora |
| Z2 | Z2–Z3 | sí | 1 | 1 < 2 → Z2→Z3 / 1 |
| Z2 | Z2–Z4 | no | — | se ignora |

Devuelve **Z2→Z3 (1)**. → U = {Z1, Z4, Z2, Z3}, VmenosU = { } (vacío). El MIENTRAS termina.

### Resumen y verificación

| Iteración | U al empezar | VmenosU al empezar | Arista elegida | Costo acum. |
|---|---|---|---|---|
| 1 | {Z1} | {Z2, Z3, Z4} | Z1–Z4 (1) | 1 |
| 2 | {Z1, Z4} | {Z2, Z3} | Z1–Z2 (2) | 3 |
| 3 | {Z1, Z4, Z2} | {Z3} | Z2–Z3 (1) | 4 |

## 3)Especificación casos de prueba

### Lenguaje natural

Cuando hay pesos repetidos puede existir más de un árbol mínimo válido. Por eso, en lugar de comparar contra una lista fija de aristas, cada prueba comprueba que el resultado cumpla las propiedades de un árbol de expansión mínimo:

- Tiene exactamente los mismos vértices que el grafo.
- Tiene V − 1 aristas.
- Es conexo: todas las zonas quedan unidas.
- No tiene ciclos.
- La suma de los pesos es igual al costo mínimo esperado.

Además se prueban los casos borde (entradas raras o extremas) y que el algoritmo no modifique el grafo que recibe.

### Tabla de casos

| # | Caso | Qué se quiere comprobar | Entrada | Resultado esperado |
|---|---|---|---|---|
| 1 | Origen inexistente | Que no falle si el origen no está en el grafo | 6 zonas, origen "Z9" | `nulo` |
| 2 | Un solo vértice | Que funcione sin ninguna arista (el MIENTRAS no se ejecuta) | solo Z1 | árbol con Z1 y 0 aristas, costo 0 |
| 3 | Dos vértices | El caso más chico con una arista | Z1–Z2 (5) | 1 arista, costo 5 |
| 4 | Ya es un árbol | Que conserve todas las aristas cuando no sobra ninguna | Z1–Z2 (1), Z2–Z3 (4), Z3–Z4 (2) | las mismas 3 aristas, costo 7 |
| 5 | Escenario reducido | El caso probado a mano en el punto 2 | zonas 1 a 4 | 3 aristas, costo 4 |
| 6 | Escenario completo | El problema del ejercicio | las 6 zonas | 5 aristas, costo 8 |
| 7 | Cualquier origen | Que el costo no dependa de la zona de inicio | 6 zonas, origen Z1 … Z6 | costo 8 en todos |

| # | Caso | Qué se quiere comprobar | Entrada | Resultado esperado |
|---|---|---|---|---|
| 8 | Empates | Que con pesos iguales dé un árbol válido | triángulo con 3 aristas de peso 1 | 2 aristas, costo 2 |
| 9 | Grafo no conexo | Que termine bien cuando `searchMinEdge` devuelve nulo | {Z1–Z2 (1)} y {Z3–Z4 (1)} separados | solo Z1–Z2; el árbol no es conexo |
| 10 | No modifica la entrada | Que el grafo original quede igual | las 6 zonas | el grafo sigue con 15 aristas y 6 vértices |

## Pseudocódigo de las pruebas

### Funciones auxiliares

```
// Verifica las propiedades que debe cumplir cualquier árbol de expansión mínimo.
verificarArbolMinimo(grafo, arbol, costoEsperado)
COMIENZO
    afirmar(arbol ≠ nulo)
    afirmar(arbol.vertices() = grafo.vertices())                      // mismos vértices
    afirmar(arbol.aristas().tamaño() = grafo.vertices().tamaño() − 1)  // V − 1 aristas
    afirmar(arbol.esConexo())                                         // todo unido
    afirmar(NO arbol.tieneCiclos())                                   // sin ciclos
    afirmar(costoTotal(arbol) = costoEsperado)                        // costo mínimo
FIN

double costoTotal(arbol)
COMIENZO
    costo ← 0
    PARA CADA e EN arbol.aristas() HACER
        costo ← costo + e.dato().getWeight()
    FIN PARA CADA
    DEVOLVER costo
FIN

// Arma el grafo completo con las primeras "cantidad" zonas de la tabla del ejercicio.
IUndirectedGraph crearGrafoZonas(cantidad)
COMIENZO
    tabla ← matriz de distancias del ejercicio (6 x 6)
    g ← nuevo UndirectedGraph<String, WeightedEdge>()
    PARA i ← 1 HASTA cantidad HACER
        g.agregarVertice("Z" + i)
    FIN PARA
    PARA i ← 1 HASTA cantidad HACER
        PARA j ← i + 1 HASTA cantidad HACER
            g.agregarArista("Z" + i, "Z" + j, nuevo WeightedEdge(tabla[i][j]))
        FIN PARA
    FIN PARA
    DEVOLVER g
FIN
```

`esConexo()`, `tieneCiclos()`, `existeArista()`, `vertices()` y `aristas()` ya existen en el `UndirectedGraph` del proyecto, así que las pruebas se pueden pasar directamente a JUnit (`afirmar` = `assertTrue` / `assertEquals`).

### Caso 1 – Origen inexistente

```
prueba01_OrigenInexistente()
COMIENZO
    g ← crearGrafoZonas(6)
    arbol ← algoritmos.prim(g, g.construirComparable("Z9"))
    afirmar(arbol = nulo)
FIN
```

### Caso 2 – Un solo vértice

```
prueba02_UnSoloVertice()
COMIENZO
    g ← nuevo UndirectedGraph<String, WeightedEdge>()
    g.agregarVertice("Z1")
    arbol ← algoritmos.prim(g, g.construirComparable("Z1"))
    verificarArbolMinimo(g, arbol, 0)
    afirmar(arbol.aristas().tamaño() = 0)
FIN
```

### Caso 3 – Dos vértices

```
prueba03_DosVertices()
COMIENZO
    g ← nuevo UndirectedGraph<String, WeightedEdge>()
    g.agregarArista("Z1", "Z2", nuevo WeightedEdge(5))
    arbol ← algoritmos.prim(g, g.construirComparable("Z1"))
    verificarArbolMinimo(g, arbol, 5)
FIN
```

### Caso 4 – El grafo ya es un árbol

```
prueba04_YaEsArbol()
COMIENZO
    g ← nuevo UndirectedGraph<String, WeightedEdge>()
    g.agregarArista("Z1", "Z2", nuevo WeightedEdge(1))
    g.agregarArista("Z2", "Z3", nuevo WeightedEdge(4))
    g.agregarArista("Z3", "Z4", nuevo WeightedEdge(2))
    arbol ← algoritmos.prim(g, g.construirComparable("Z1"))
    verificarArbolMinimo(g, arbol, 7)
    PARA CADA e EN g.aristas() HACER                // están todas las aristas originales
        afirmar(arbol.existeArista(g.construirComparable(e.source()),
                                   g.construirComparable(e.target())))
    FIN PARA CADA
FIN
```

### Caso 5 – Escenario reducido (4 zonas)

```
prueba05_EscenarioReducido()
COMIENZO
    g ← crearGrafoZonas(4)
    arbol ← algoritmos.prim(g, g.construirComparable("Z1"))
    verificarArbolMinimo(g, arbol, 4)
FIN
```

### Caso 6 – Escenario completo (6 zonas)

```
prueba06_EscenarioCompleto()
COMIENZO
    g ← crearGrafoZonas(6)
    arbol ← algoritmos.prim(g, g.construirComparable("Z1"))
    verificarArbolMinimo(g, arbol, 8)
FIN
```

### Caso 7 – Cualquier origen da el mismo costo

```
prueba07_CualquierOrigen()
COMIENZO
    g ← crearGrafoZonas(6)
    PARA i ← 1 HASTA 6 HACER
        arbol ← algoritmos.prim(g, g.construirComparable("Z" + i))
        verificarArbolMinimo(g, arbol, 8)
    FIN PARA
FIN
```

## Caso 8 – Empates de pesos

```
prueba08_Empates()
COMIENZO
    g ← nuevo UndirectedGraph<String, WeightedEdge>()
    g.agregarArista("Z1", "Z2", nuevo WeightedEdge(1))
    g.agregarArista("Z2", "Z3", nuevo WeightedEdge(1))
    g.agregarArista("Z1", "Z3", nuevo WeightedEdge(1))
    arbol ← algoritmos.prim(g, g.construirComparable("Z1"))
    verificarArbolMinimo(g, arbol, 2)               // cualquier par de aristas es válido
FIN
```

## Caso 9 – Grafo no conexo

```
prueba09_NoConexo()
COMIENZO
    g ← nuevo UndirectedGraph<String, WeightedEdge>()
    g.agregarArista("Z1", "Z2", nuevo WeightedEdge(1))
    g.agregarArista("Z3", "Z4", nuevo WeightedEdge(1))
    arbol ← algoritmos.prim(g, g.construirComparable("Z1"))
    afirmar(arbol ≠ nulo)
    afirmar(arbol.aristas().tamaño() = 1)           // solo la componente de Z1
    afirmar(arbol.existeArista(g.construirComparable("Z1"), g.construirComparable("Z2")))
    afirmar(NO arbol.esConexo())
FIN
```

## Caso 10 – No modifica el grafo original

```
prueba10_NoModificaEntrada()
COMIENZO
    g ← crearGrafoZonas(6)
    aristasAntes ← g.aristas().tamaño()             // 15
    algoritmos.prim(g, g.construirComparable("Z1"))
    afirmar(g.aristas().tamaño() = aristasAntes)
    afirmar(g.vertices().tamaño() = 6)
FIN
```

# 4)Póster

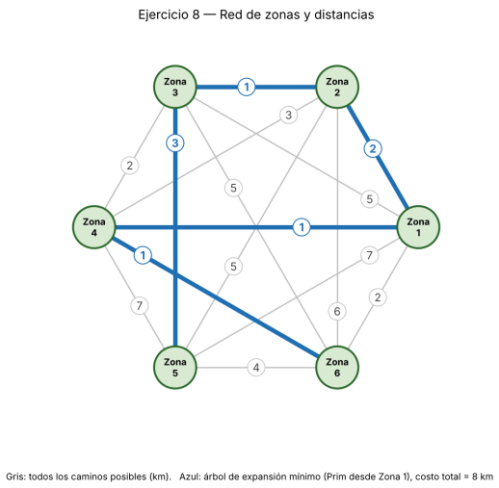
# Caminos de reforestación con el algoritmo de Prim

Árbol de expansión mínimo · Algoritmos y Estructuras de Datos · UCU · Problem Set 04 – Ejercicio 8

## El problema

Una empresa de reforestación debe unir **6 zonas** con caminos de tierra, de forma que se pueda ir de cualquier zona a cualquier otra, construyendo la **menor cantidad total de km**.

Se modela como un **grafo no dirigido y ponderado**: cada zona es un vértice, cada camino posible es una arista y su peso son los km.




Resultado: **1–4, 4–6, 1–2, 2–3, 3–5 → 8 km**

## La idea de Prim

- Empezar con una sola zona en el árbol (conjunto **U**).
- En cada paso, agregar el camino **más corto** que une una zona de U con una que **falta** (V − U).
- Repetir hasta conectar todas: siempre quedan **V − 1** caminos y ningún ciclo.

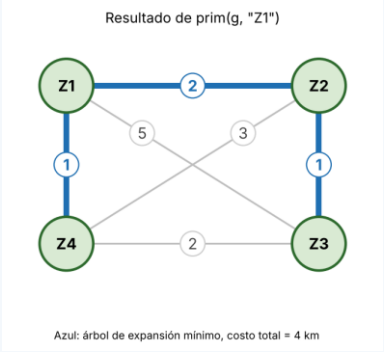
**Orden:** O(V · E) con `searchMinEdge`; O(E log V) con cola de prioridad.

## Pseudocódigo (TDAs del proyecto)

```
prim(grafo, origen)
COMIENZO
  inicio ← grafo.buscarVertice(origen)
  SI inicio = nulo ENTONCES DEVOLVER nulo
  U ← {inicio}
  VmenosU ← grafo.vertices() − {inicio}
  arbol ← nuevo UndirectedGraph con los vértices de grafo
  MIENTRAS NO VmenosU.esVacio() HACER
    arista ← searchMinEdge(grafo, U, VmenosU)
    SI arista = nulo ENTONCES SALIR   // no conexo
    arbol.agregarArista(arista.source(),
                        arista.target(), arista.dato())
    U.agregar(arista.target())
    VmenosU.remover(arista.target())
  FIN MIENTRAS
  DEVOLVER arbol
FIN

searchMinEdge(grafo, U, V)
COMIENZO
  mejor ← nulo ; mejorPeso ← +∞
  PARA CADA u EN U HACER
    PARA CADA e EN grafo.adyacencias(u) HACER
      w ← e.target()
      SI V.contiene(w) Y e.dato().getWeight() < mejorPeso
        mejor ← arista (u → w) ; mejorPeso ← peso de e
  DEVOLVER mejor
FIN
```

## Prueba con 4 zonas




| It. | U | Elegida | Costo |
|---|---|---|---|
| 1 | {Z1} | Z1–Z4 (1) | 1 |
| 2 | {Z1, Z4} | Z1–Z2 (2) | 3 |
| 3 | {Z1, Z4, Z2} | Z2–Z3 (1) | 4 |

## Casos de prueba

Cada prueba verifica: mismos vértices, **V − 1** aristas, conexo, sin ciclos y **costo esperado**.

| # | Caso | Esperado |
|---|---|---|
| 1 | Origen inexistente | nulo |
| 2 | Un solo vértice | 0 aristas |
| 3 | Dos vértices | 1 arista |
| 4 | Ya es un árbol | el mismo grafo |
| 5 | Escenario de 4 zonas | costo 4 |
| 6 | Escenario de 6 zonas | costo 8 |
| 7 | Origen Z1 … Z6 | siempre costo 8 |
| 8 | Empates de pesos | árbol válido |
| 9 | Grafo no conexo | solo la componente |
| 10 | No modifica la entrada | 15 aristas intactas |